

OA Dev Guide for Eclipse

Contents

Introduction	1
Software Development principals.....	2
Creating a new Project.....	3
Source code.....	3
Create Eclipse Project	3
Create a new OABuilder Model	4
Model setup	8
Run Code Generation for first time	9
Configure Eclipse as a Maven project.....	11
Change the Java Build Path.....	12
Run Application.....	14
Run Application from Eclipse (first time).....	14
Run as Server	20
Run as Client	23
Reviewing what was generated	27
Create new Version.....	32
Adding project to the OAppStore	32

Date	Version	Description	User	Note
May 18, 2024	1.0.x	Initial Document	Vince Via	

Introduction

This document will step through the creation of a new Software Product using OABuilder, Eclipse (Java IDE) and then running.

Software Development principals

Follows main principals behind *“software that develops software”*. Tools and libraries of designers, modelers, reverse and forward engineering, code generation, high-reuse libraries, full stack automation solutions.

Model Driven Development

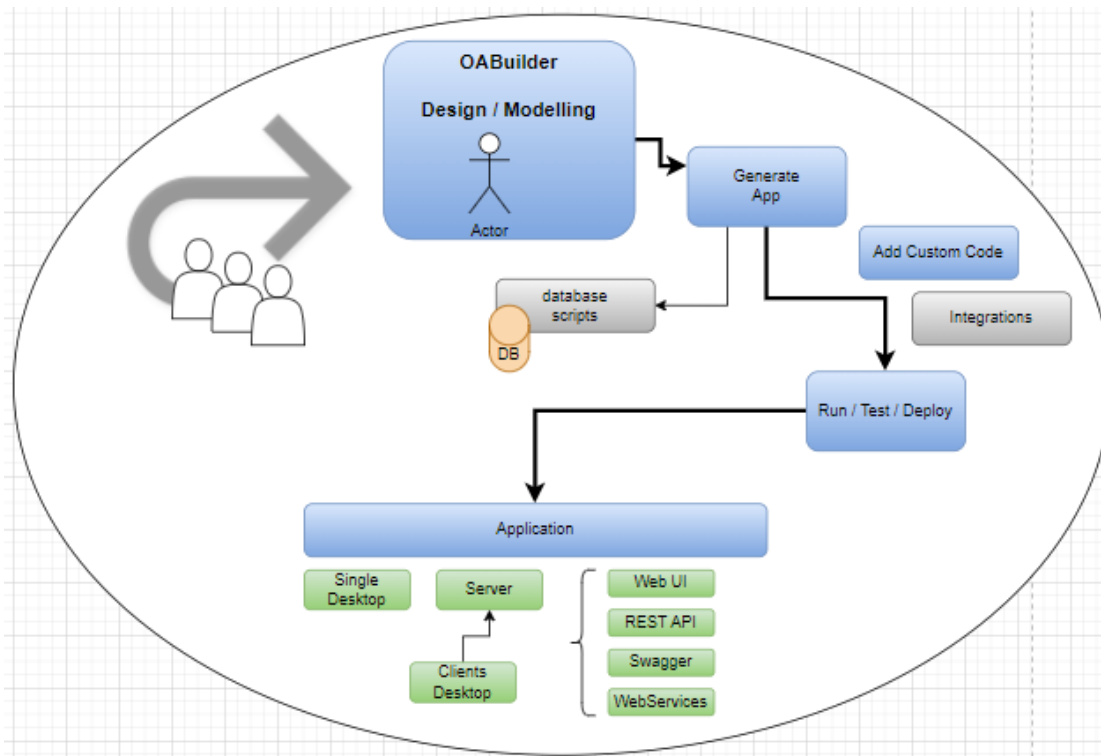
“The Model is the Application”

“The Model drives the development”

“The Model is the Product’s Blueprint”

Commercial grade products with full stack support, security, monitoring, etc.

Agile, Iterative process.



See Guides for further information on libraries and development tools.

OA-Core – github, javadoc

OA-JFC – github, javadoc

OA-Web – github, javadoc

OAApStore – Application Store for running all OA and Java applications - installer, User Guide

OABuilder – software development modeler and application generator - online help

See OAApStore user guide for installing OABuilder.

Creating a new Project

Source code

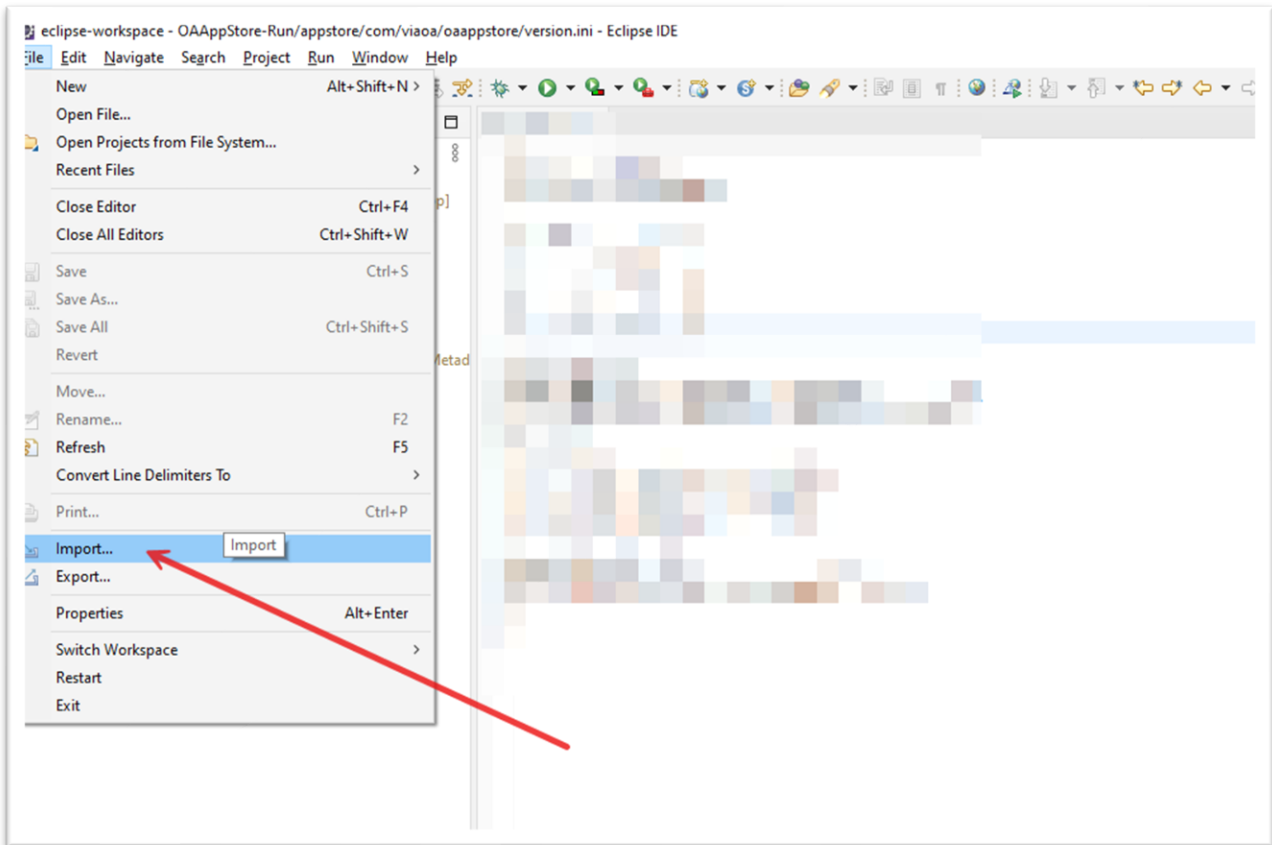
Create a new project in Repo.

Example: use github, bitbucket, ADO, etc.

Create Eclipse Project

Clone the project from repo (copy to clipboard).

Import your project from Repo.

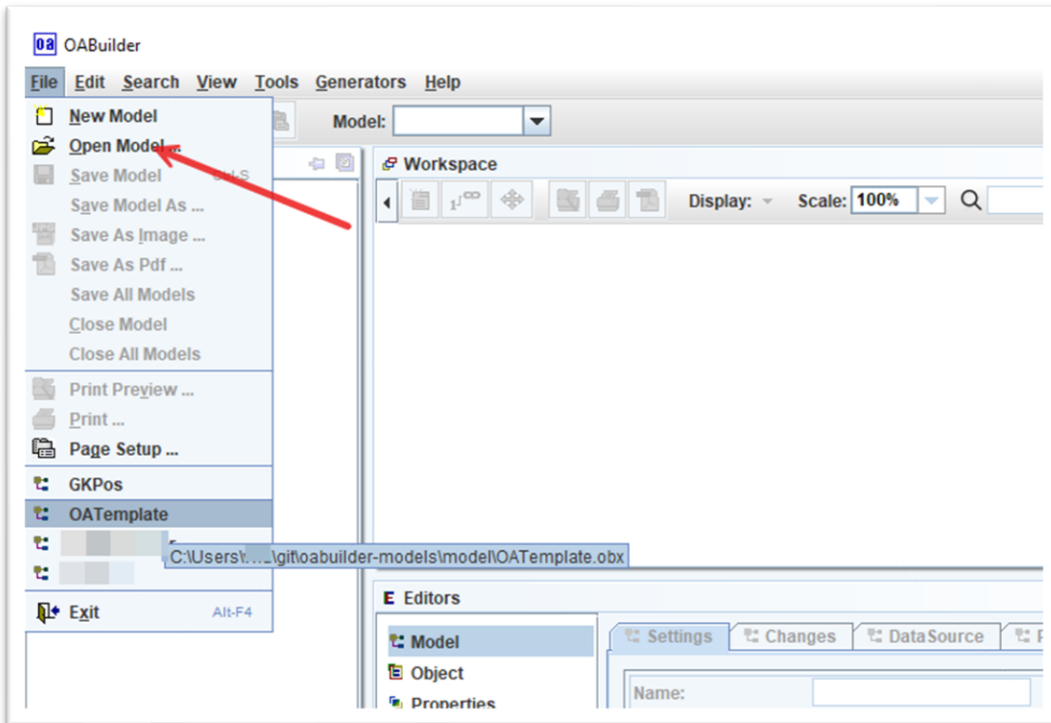


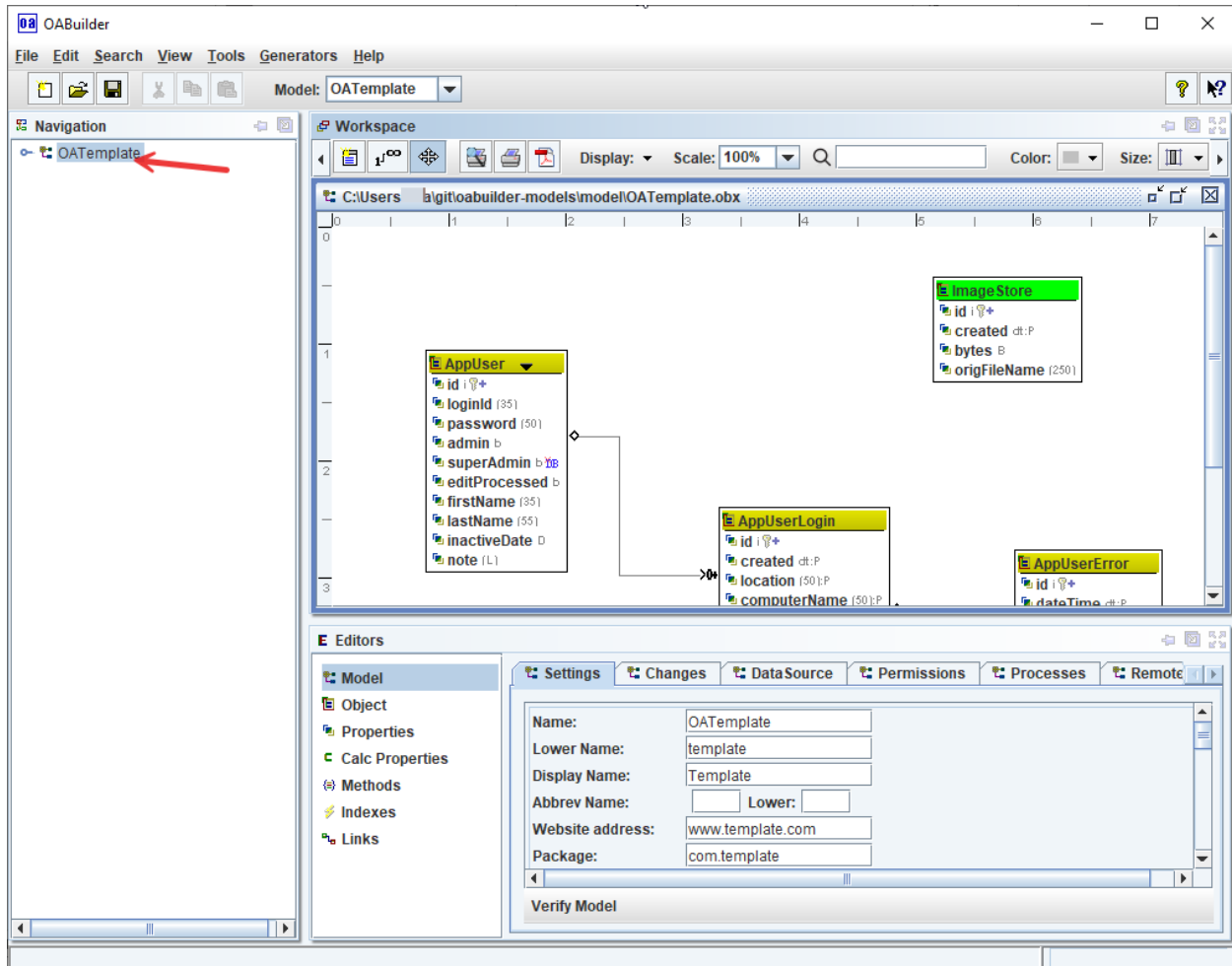
Do not do any additional configuration at this time.

Create a new OABuilder Model

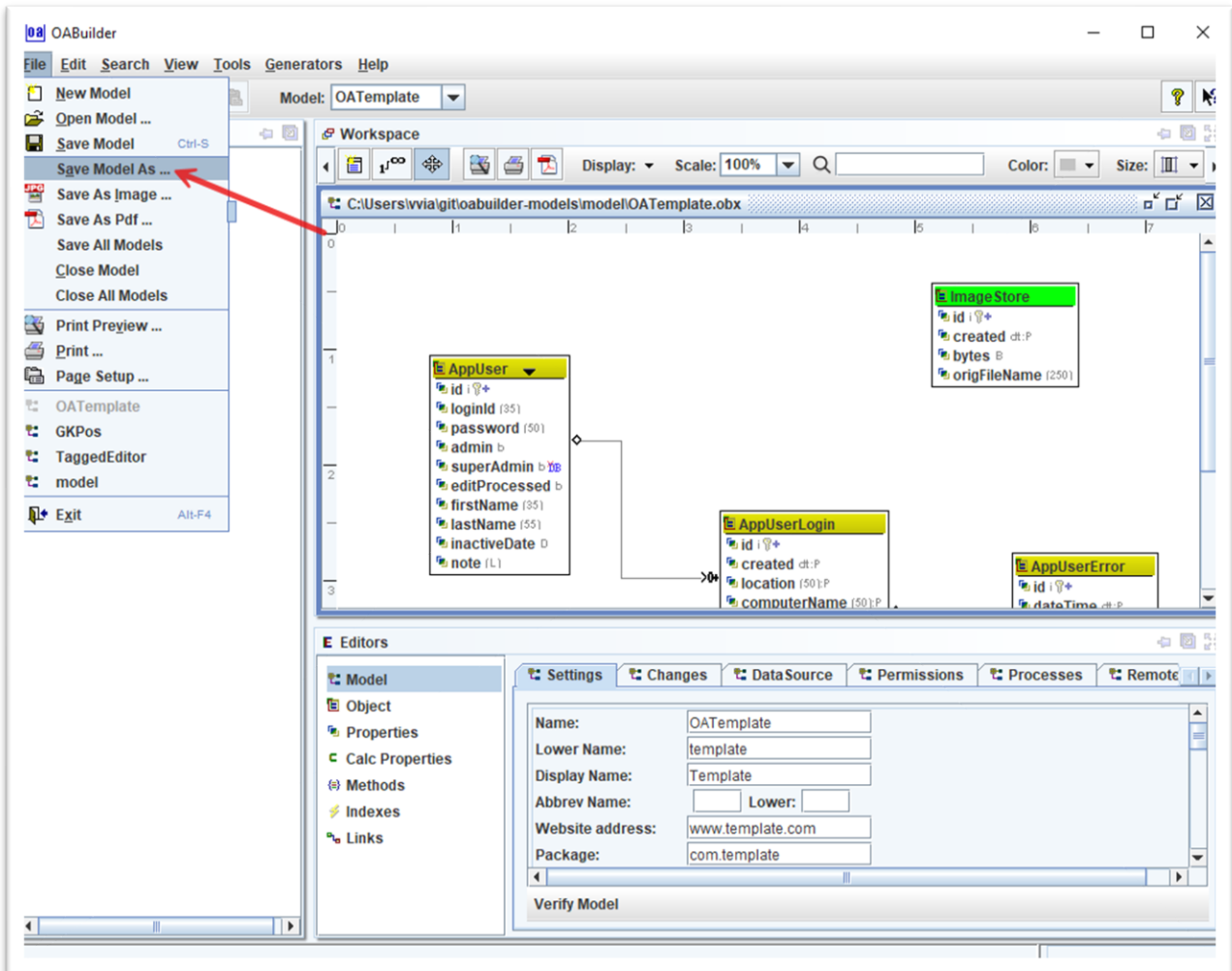
Run OABuilder

Open model "OATemplate.obx" (located in the install directory, or get it from github OABuilder-models)

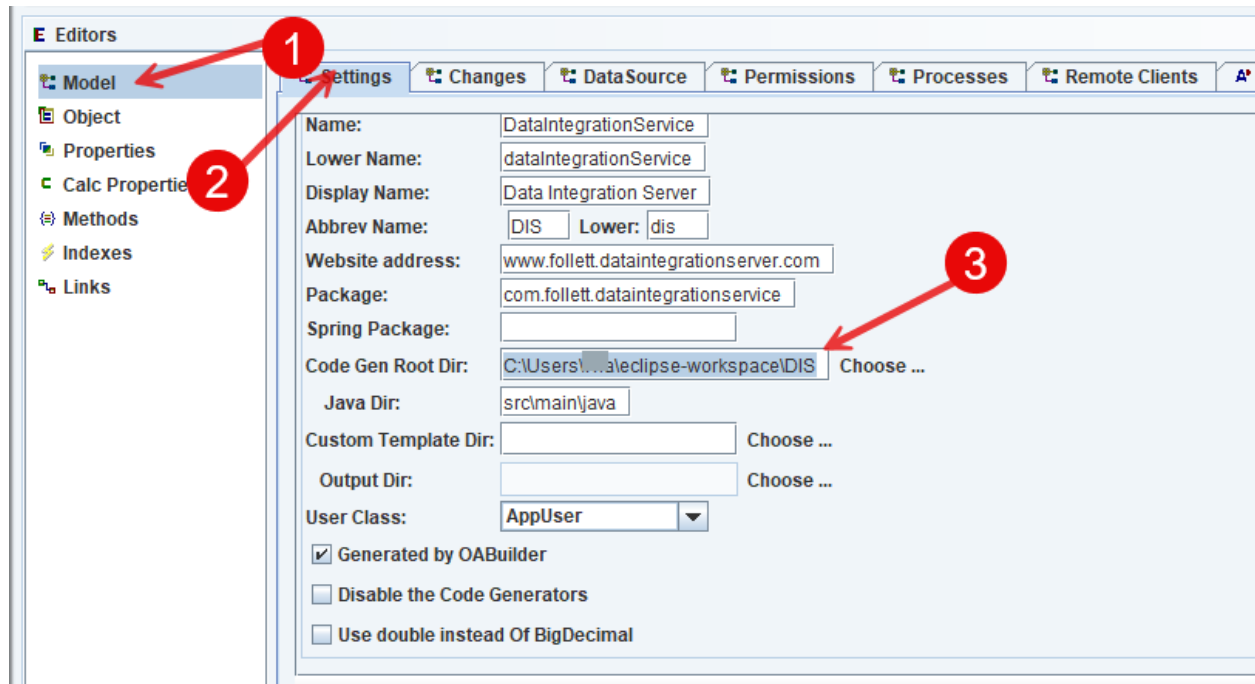




Choose “Save as” from the “File” menu, and use your project name as the new Model’s name.

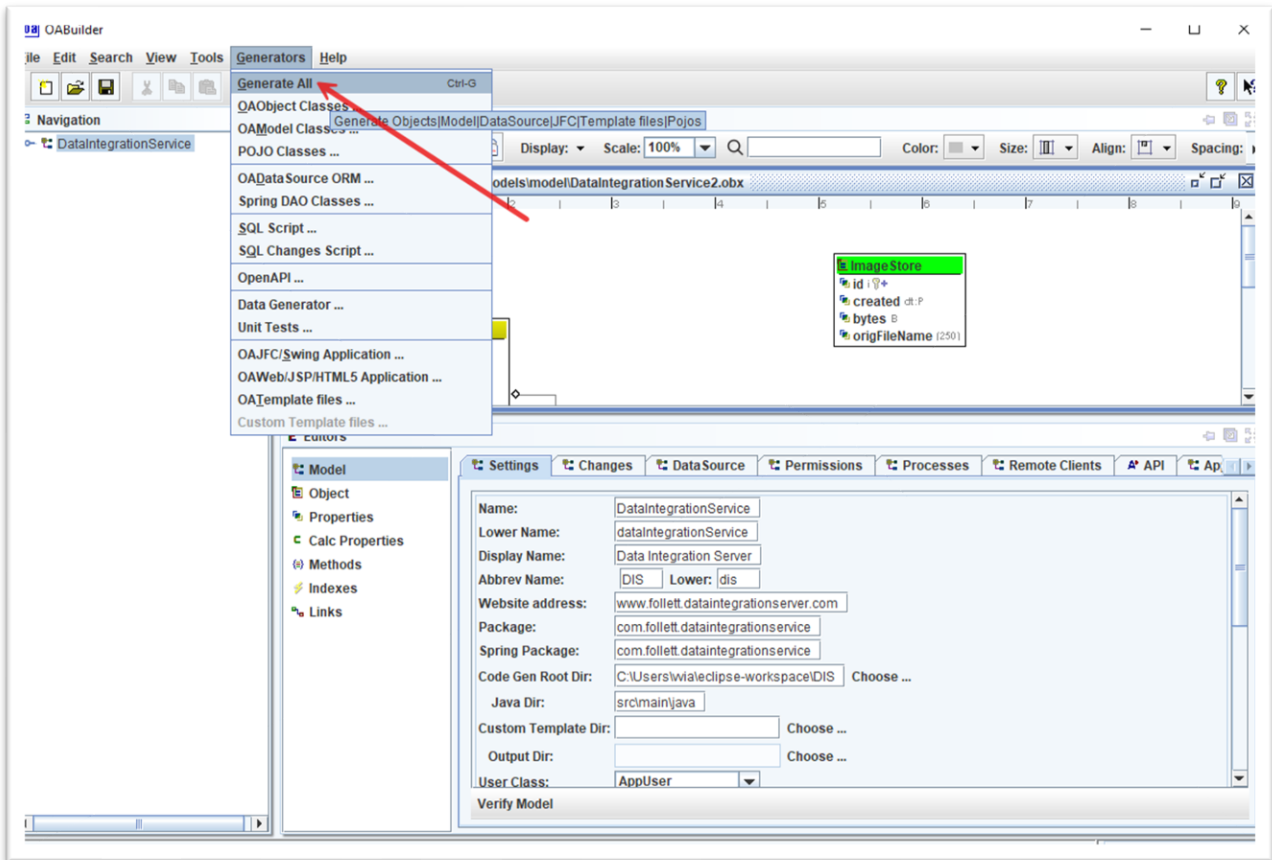


Model setup

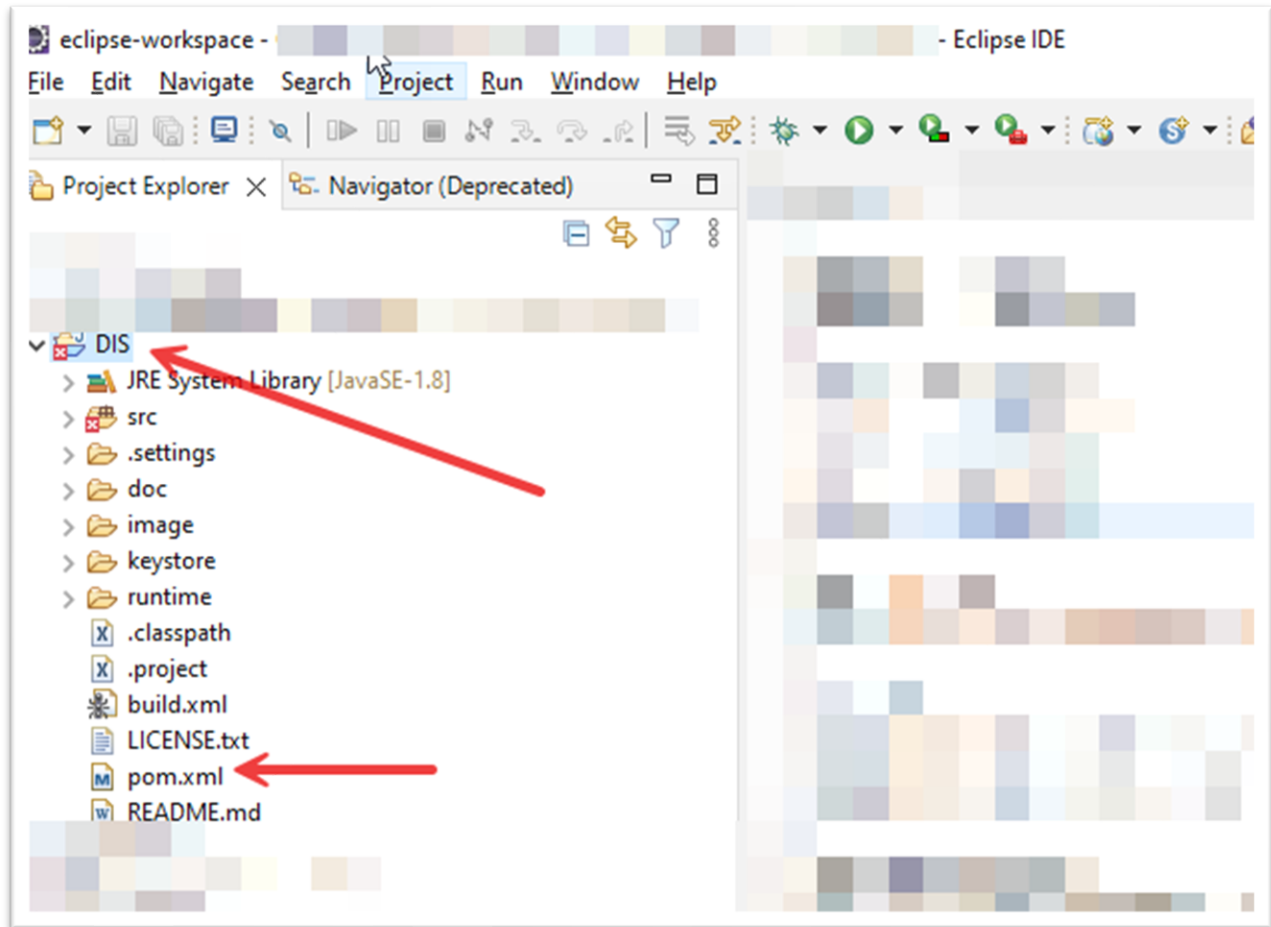


1. Go to "Model" Editor
2. Select "Settings" tab
3. Set the "Code Gen Root Dir" to the directory for your project.

Run Code Generation for first time

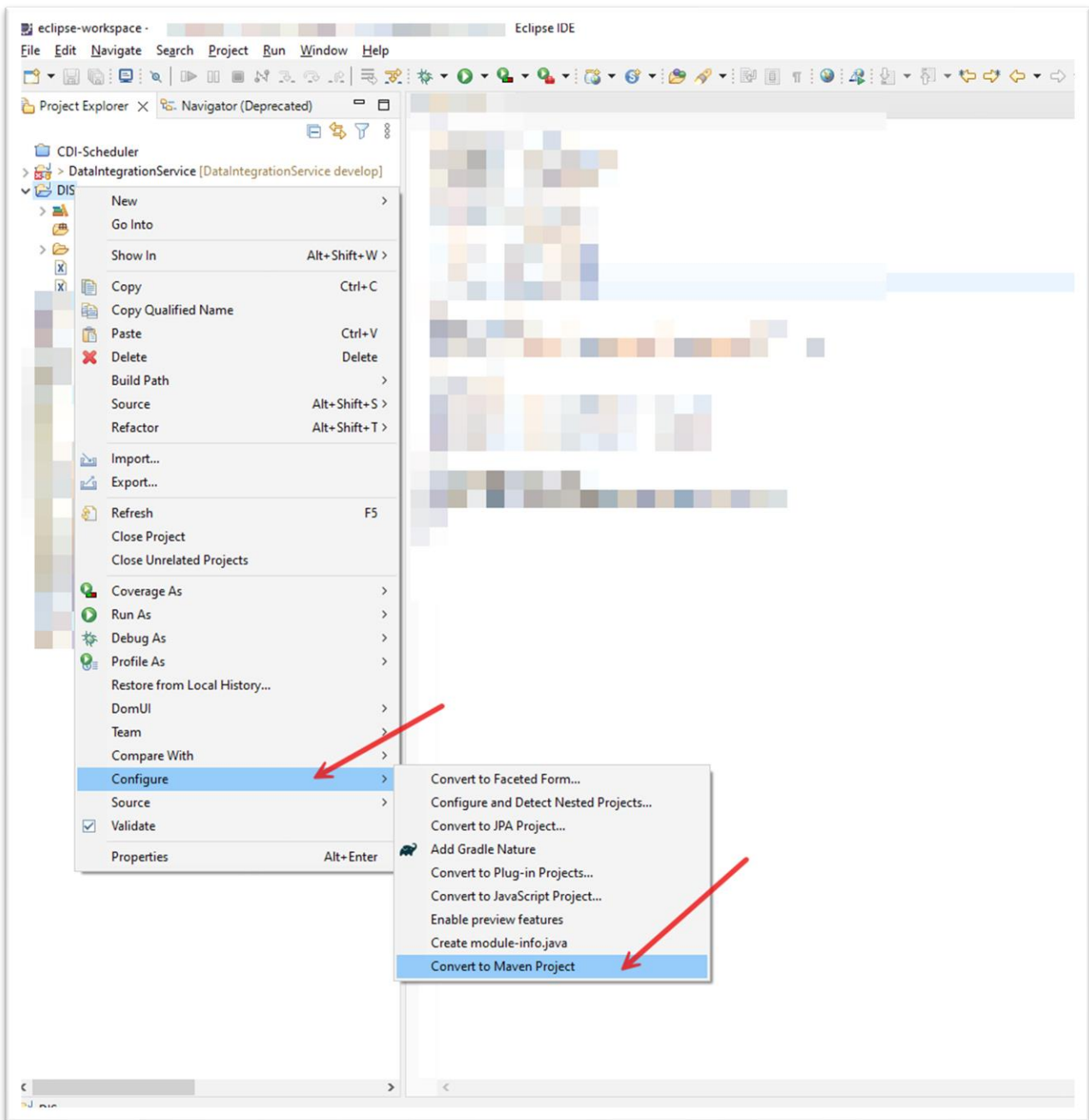


Eclipse now has code

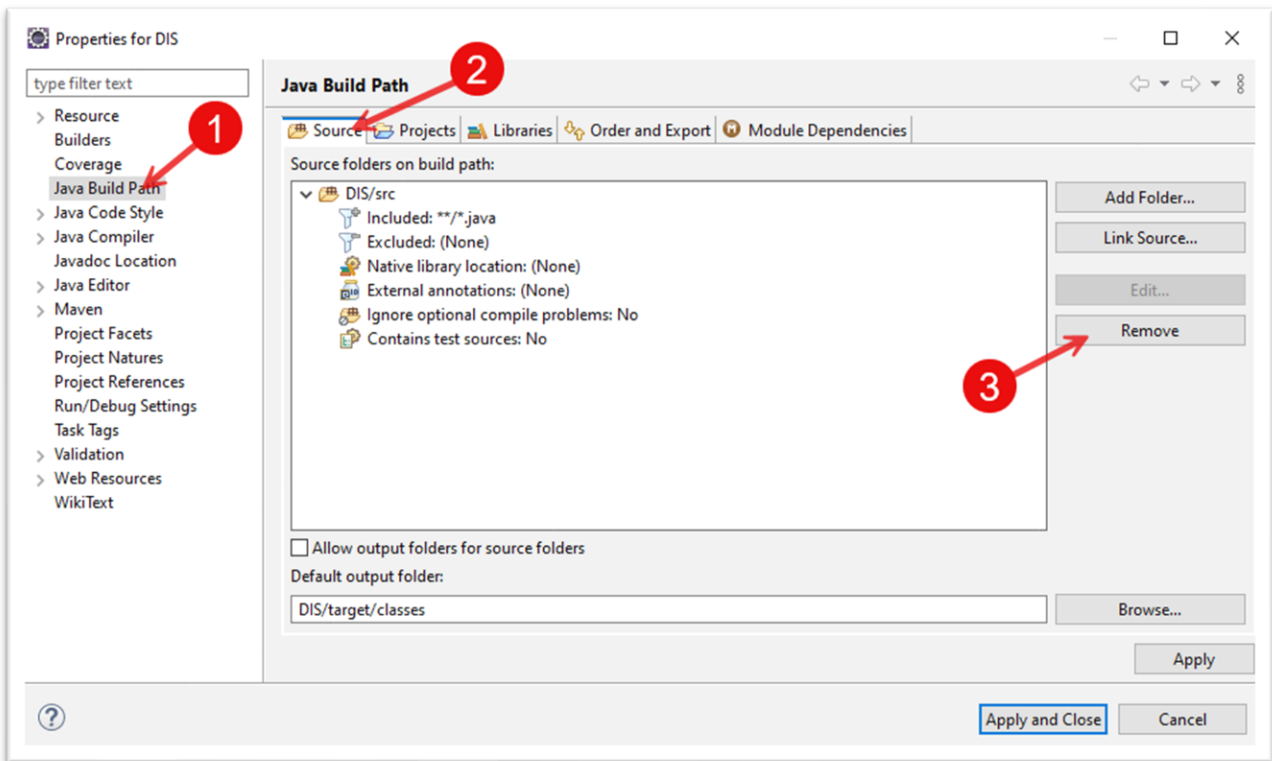


Refresh the project in Eclipse.

Configure Eclipse as a Maven project



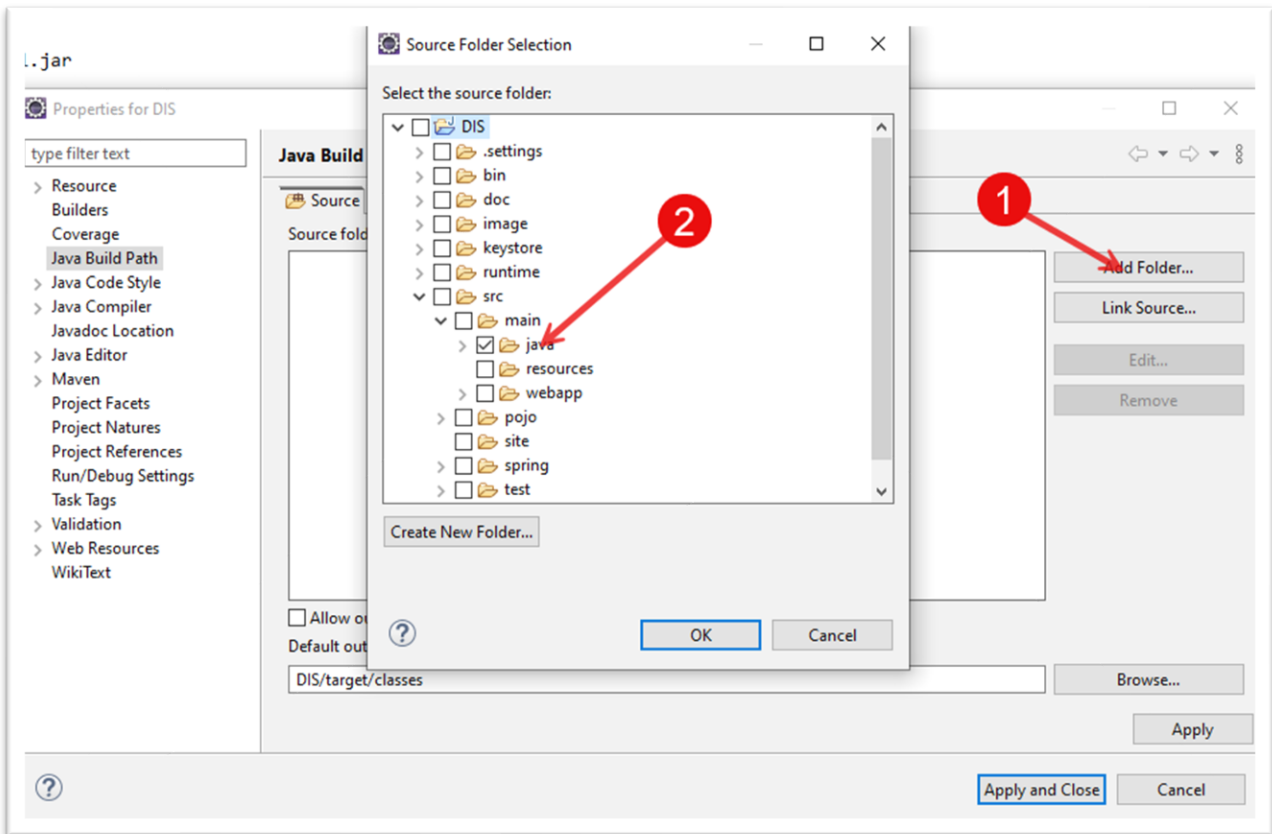
Change the Java Build Path.



Right click the project and select “project properties”. Then:

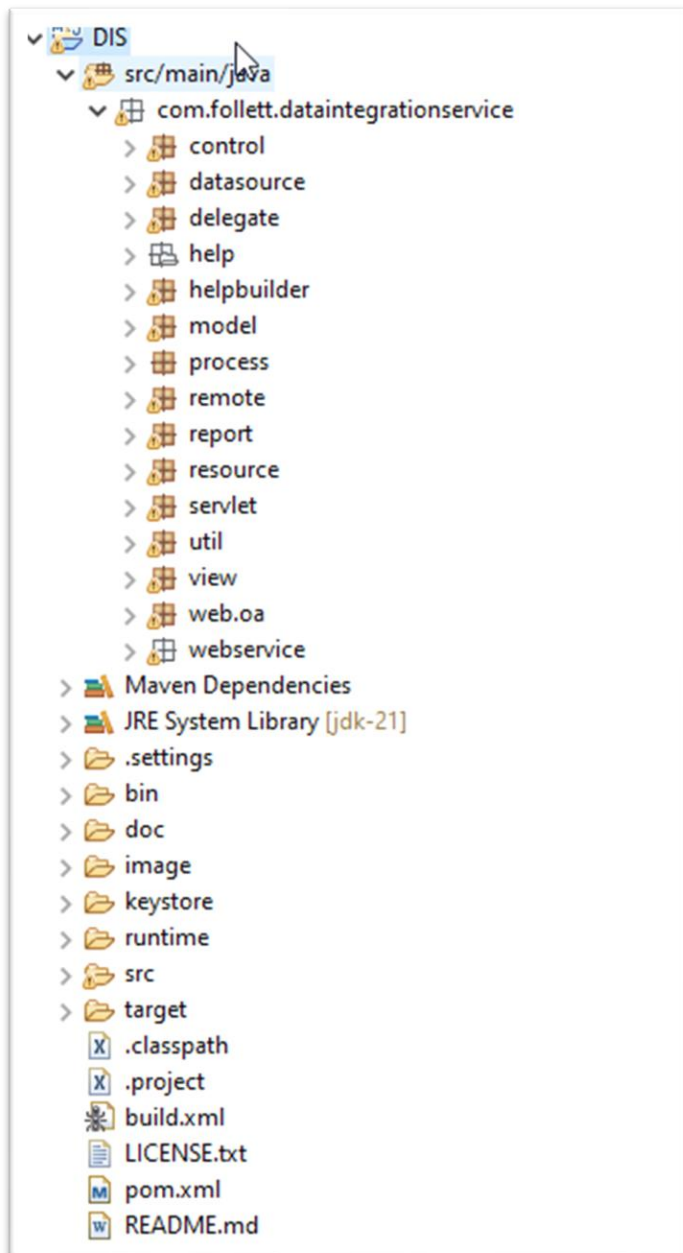
1. Select “Java Build Path”
2. Select “Source” tab
3. Click “Remove” to remove the current “source folder” location.

Set the “Source Folder” to match what Maven uses.



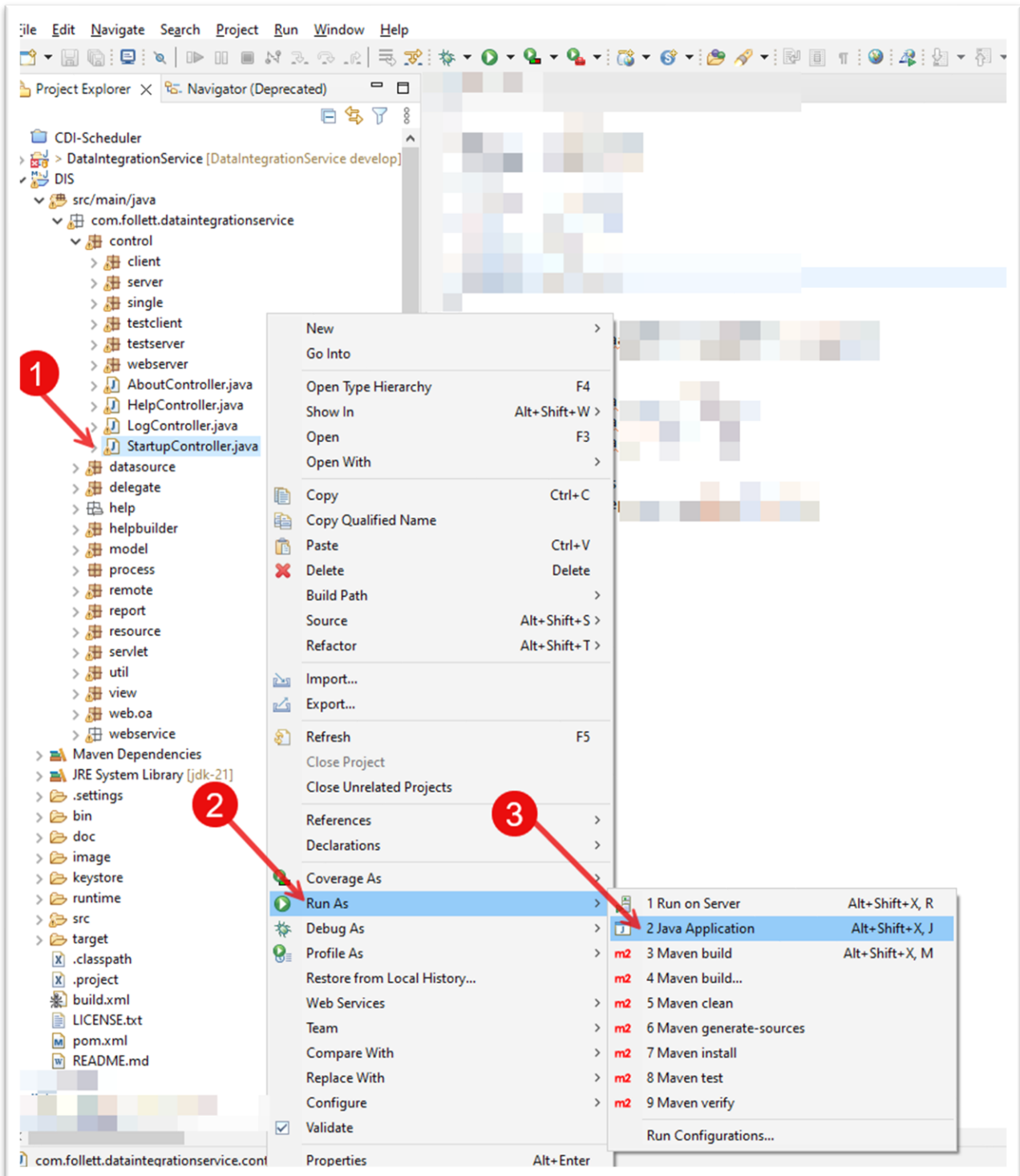
1. Click Add Folder button to choose the source code directory.
2. Check “src/main/java”

Run Application



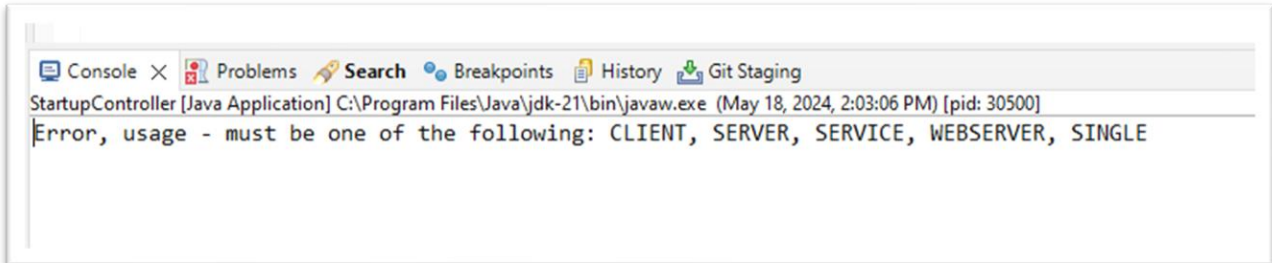
The code was generated and the project is free from any errors.

Run Application from Eclipse (first time)

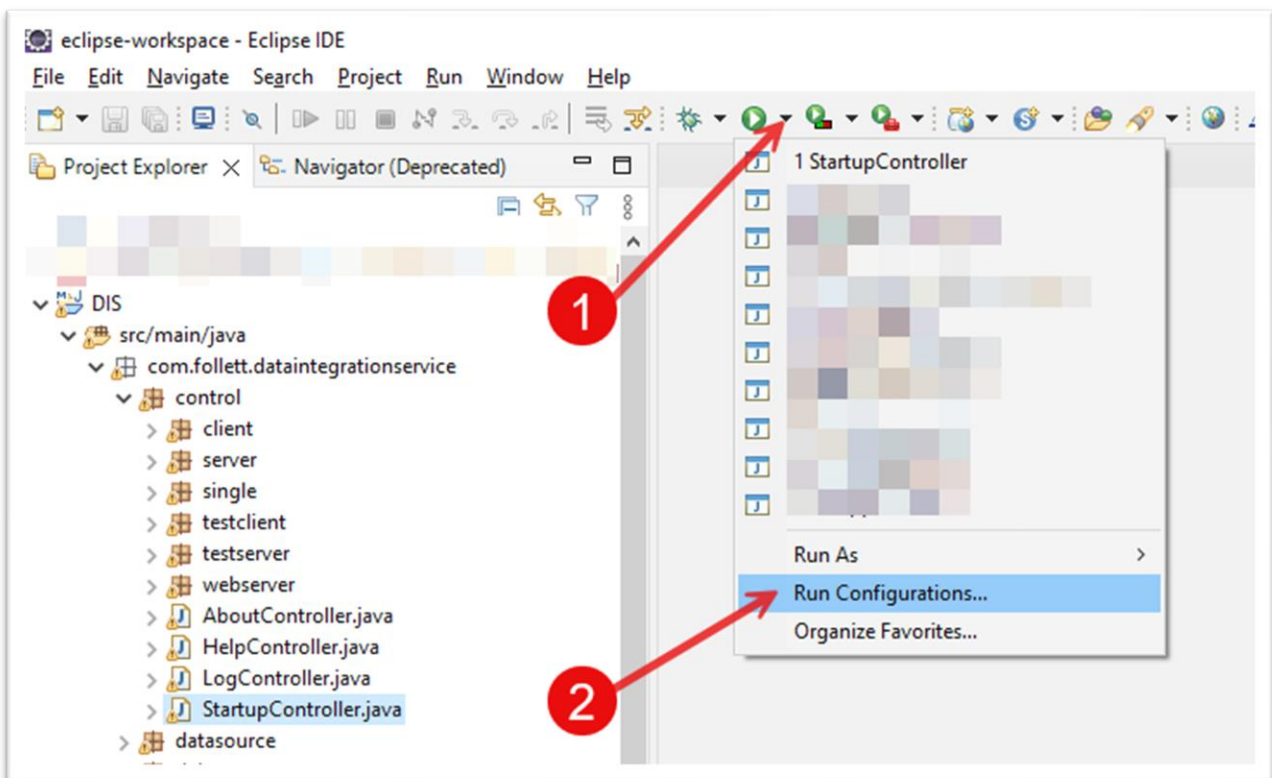


1. Select "StartupController.java"
2. Right click, and select "Run As" from the menu.
3. Select "Java Application" from secondary menu.

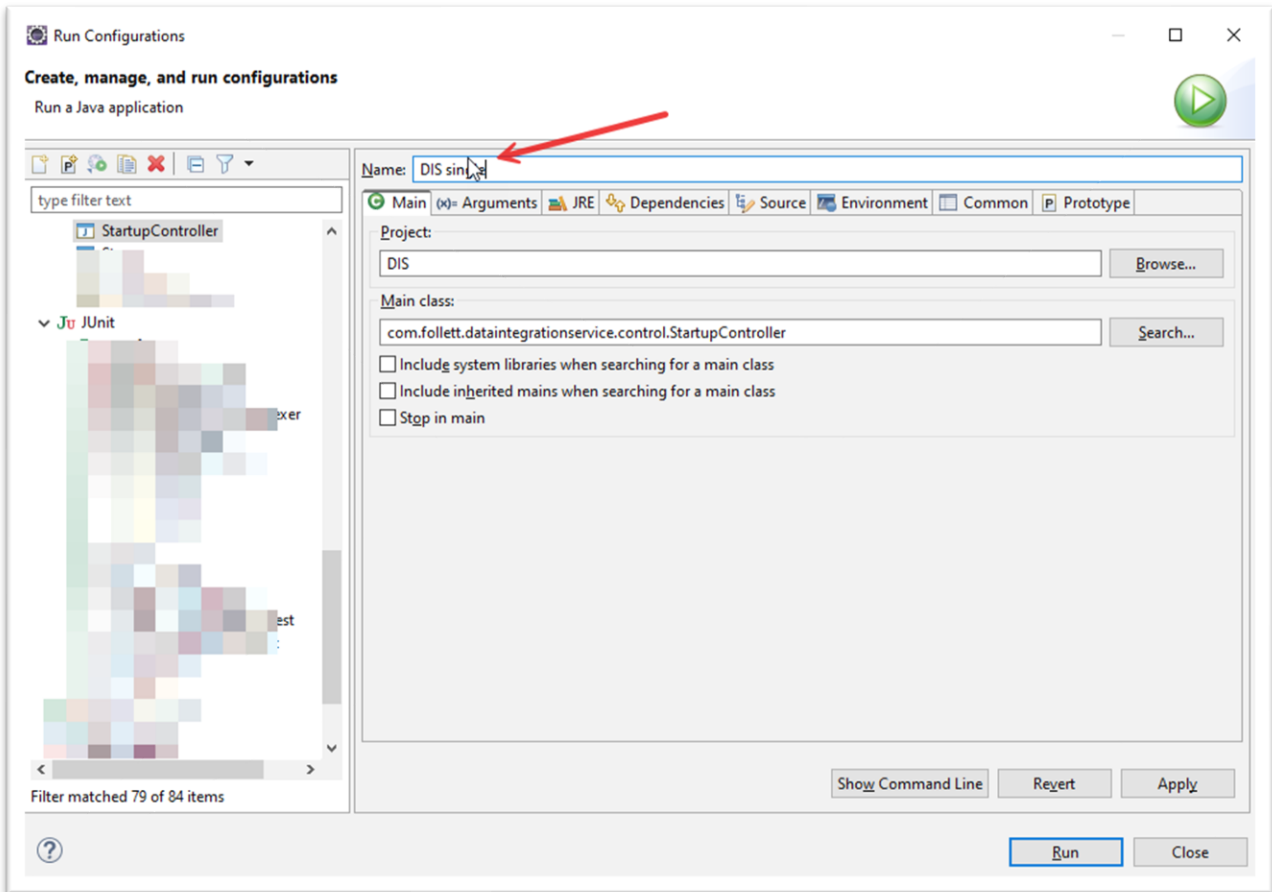
This will run the application, but will exit with an error message in the console.



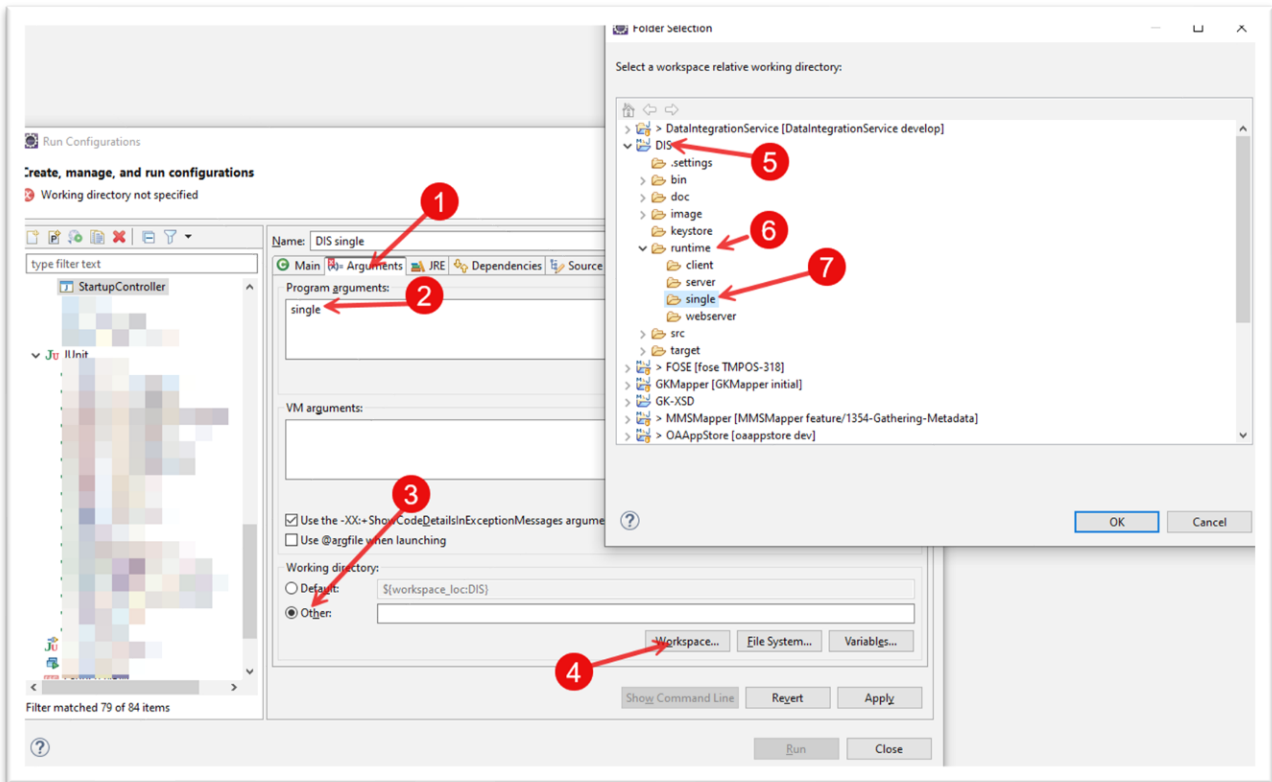
The next step is to make a few changes to the “run” configuration.



1. Click “run” down-arrow button on main toolbar
2. Select “Run Configurations”

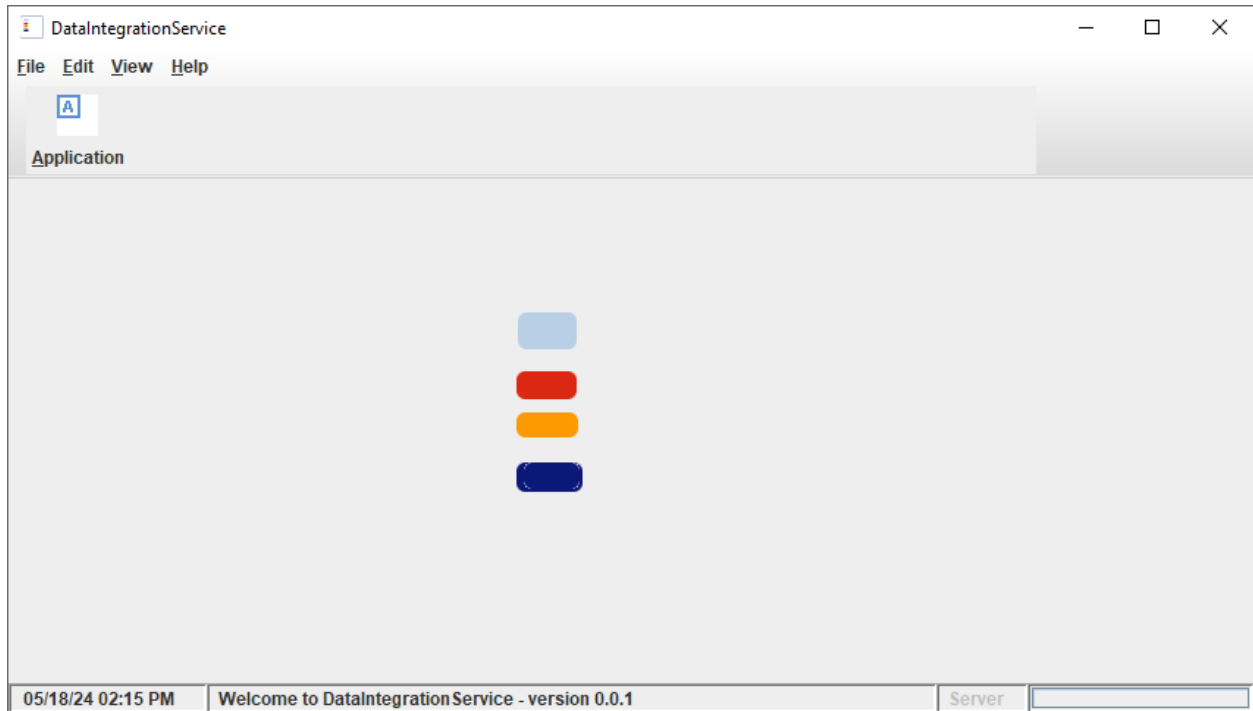


Change the name to project name + “single” since we’re first going to run it as a “single user” app.



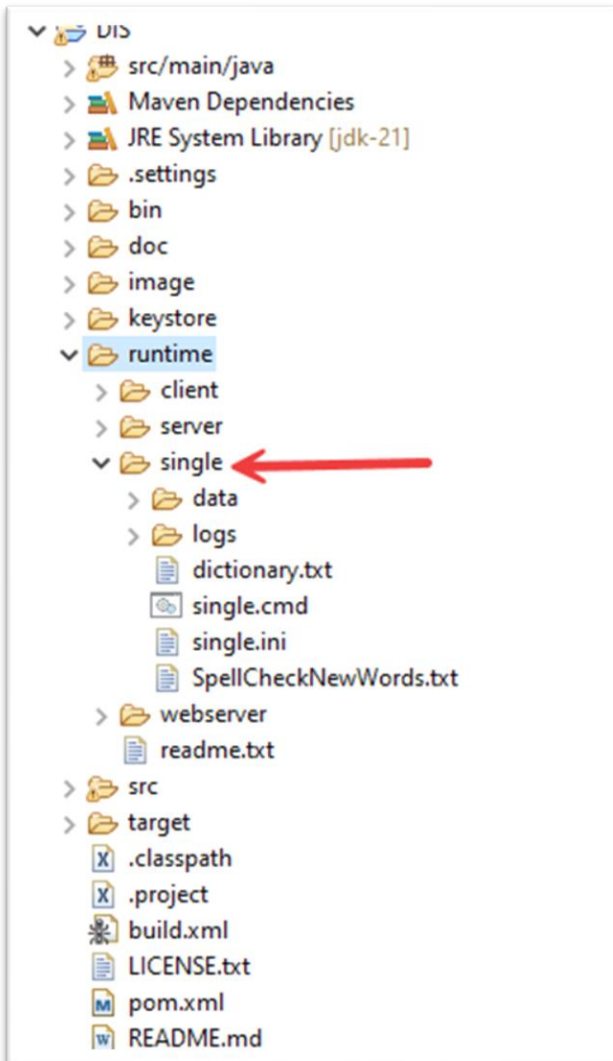
1. Go to arguments tab
2. Add "single" to program arguments.
3. Set working directory to "other"
4. Find your project name
5. Expand to the "runtime" directory
6. Choose single, this is where the program will be running from.

Select "Ok" and then click the "run"



The app is now running as a single user application.

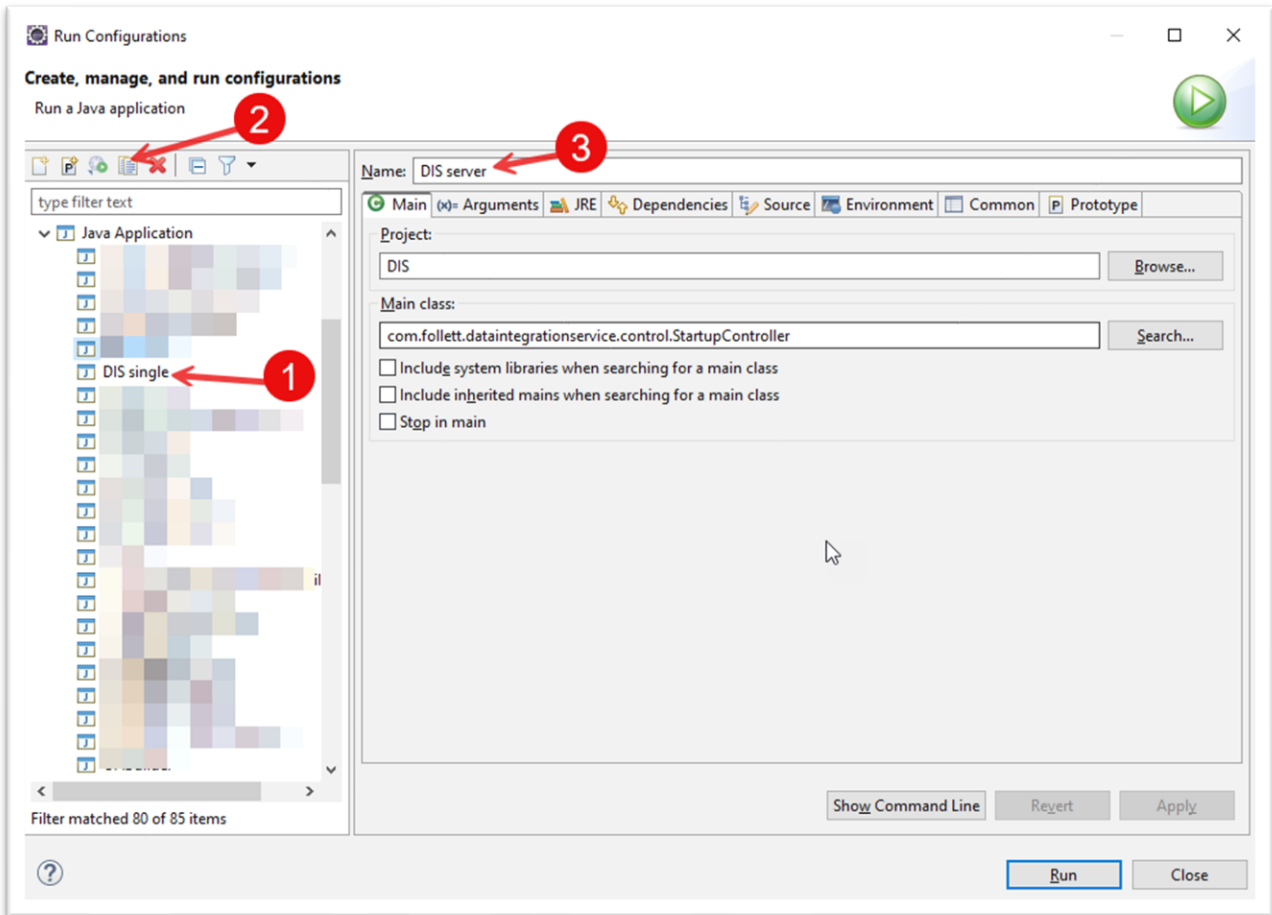
By default, the app is running in the "runtime/single" directory, and there is a subdirectory "data" for storing data (no database used at this time), the logs directory, and a single.ini file with properties.



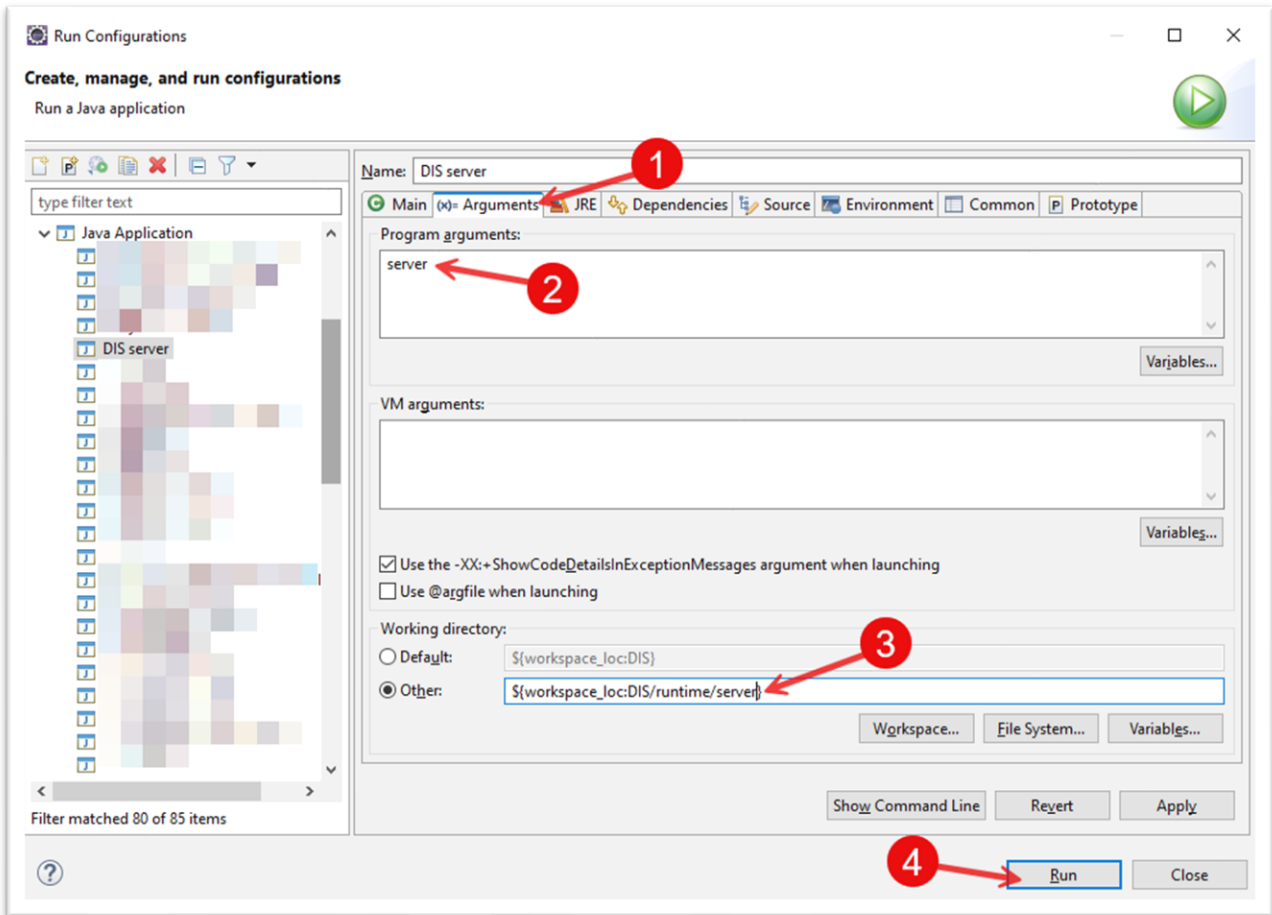
You will need to refresh the Eclipse project to see latest changes that are made outside of Eclipse.

Run as Server

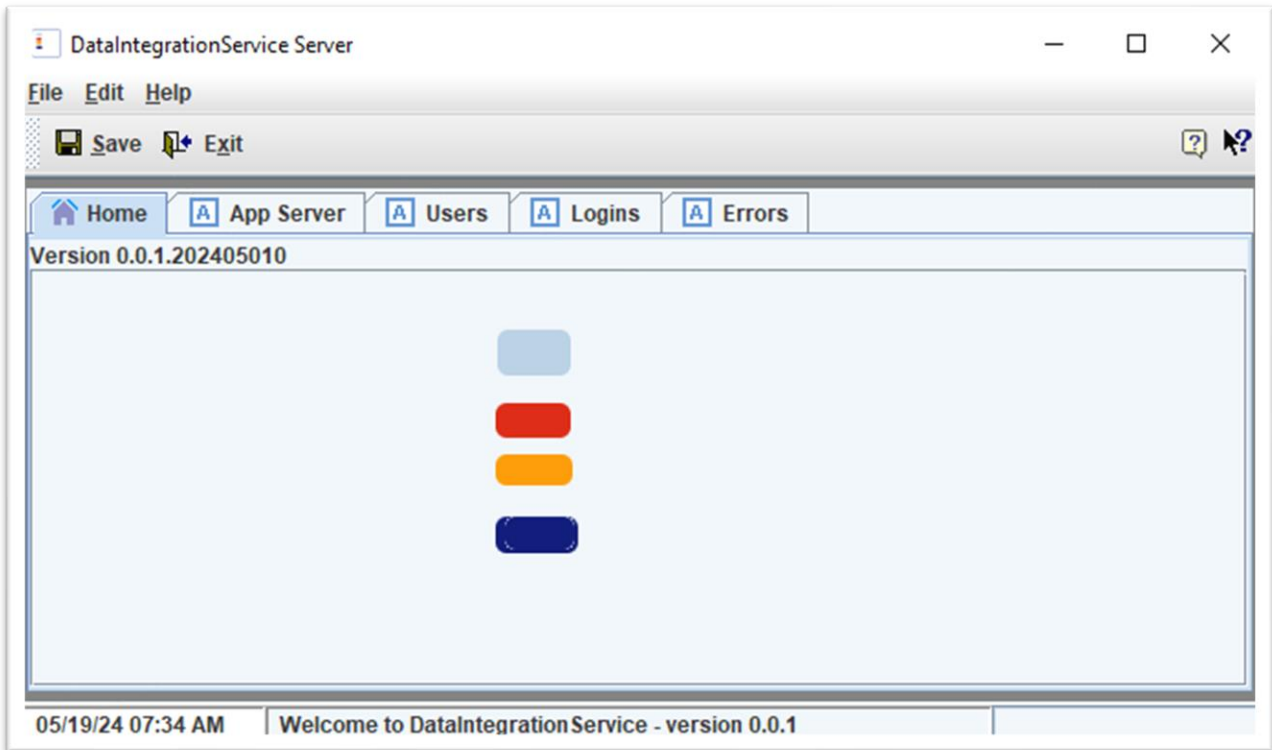
Click on the menu for run configurations.



1. Select the project.
2. Click the “copy” button, which will create a new run configuration.
3. Rename to “server”.



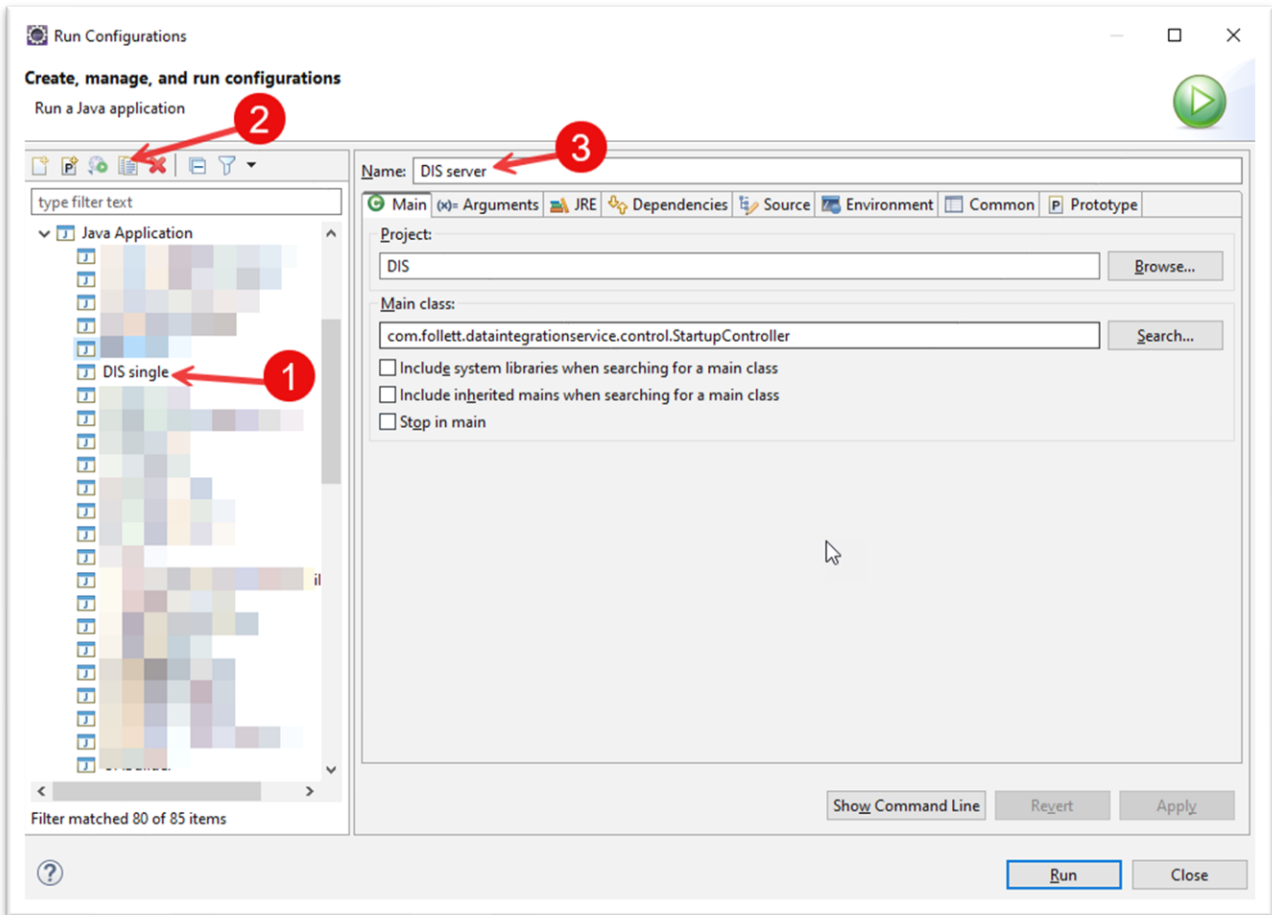
1. Click the "Arguments" tab
2. Change to "server"
3. Change from "single" to "server"
4. Click "run" button



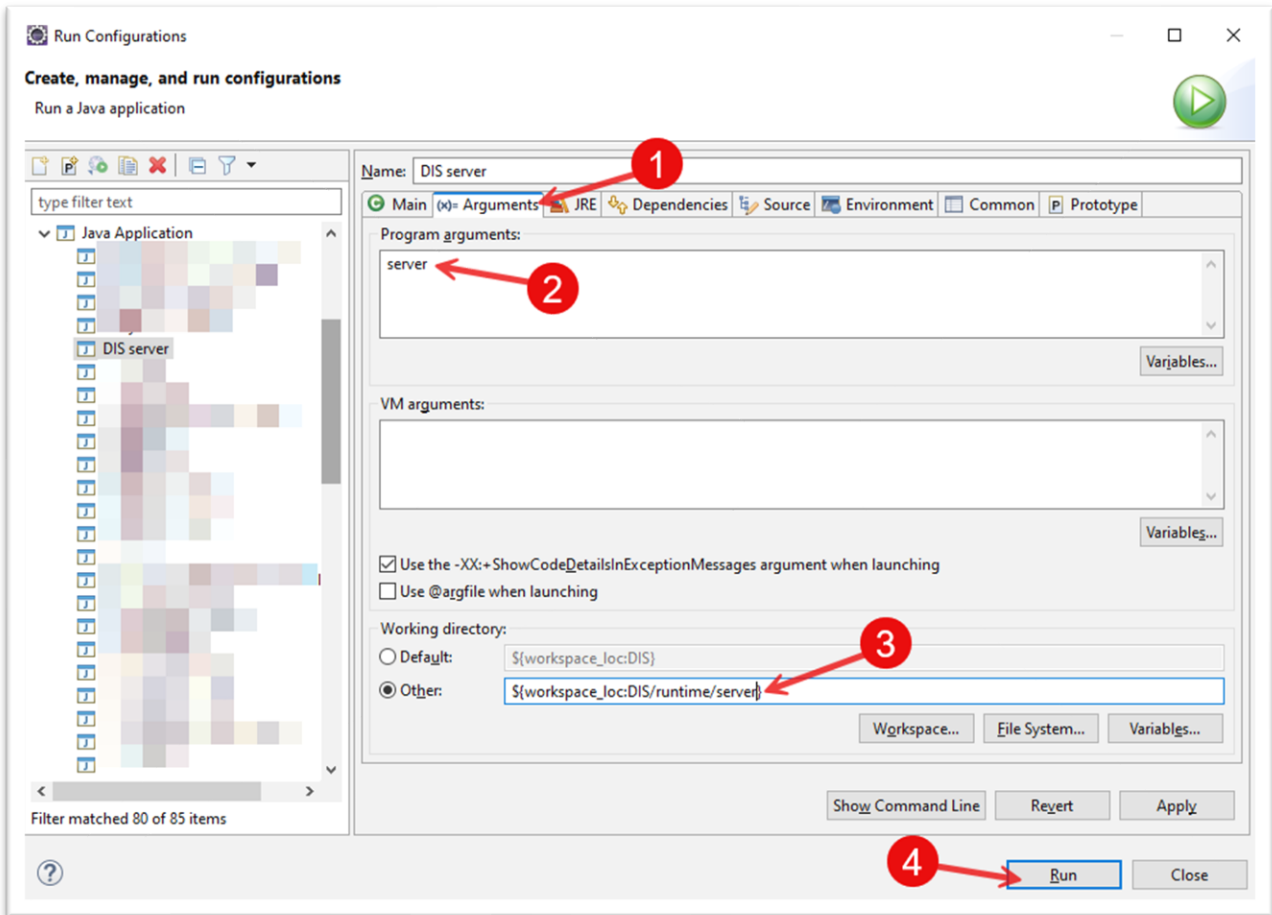
Application running as a server.

Run as Client

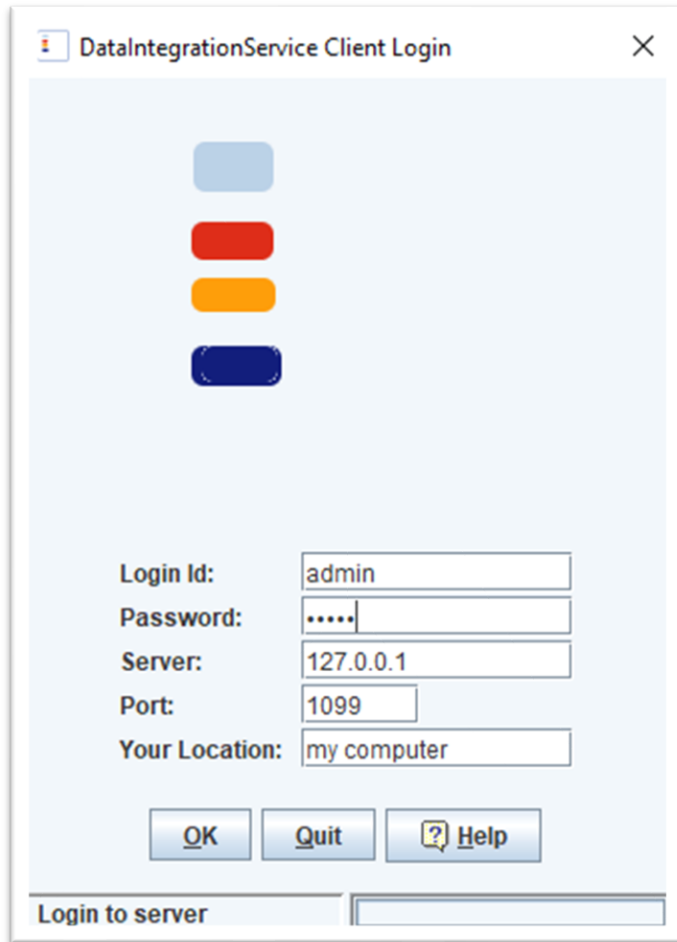
Click on the menu for run configurations.



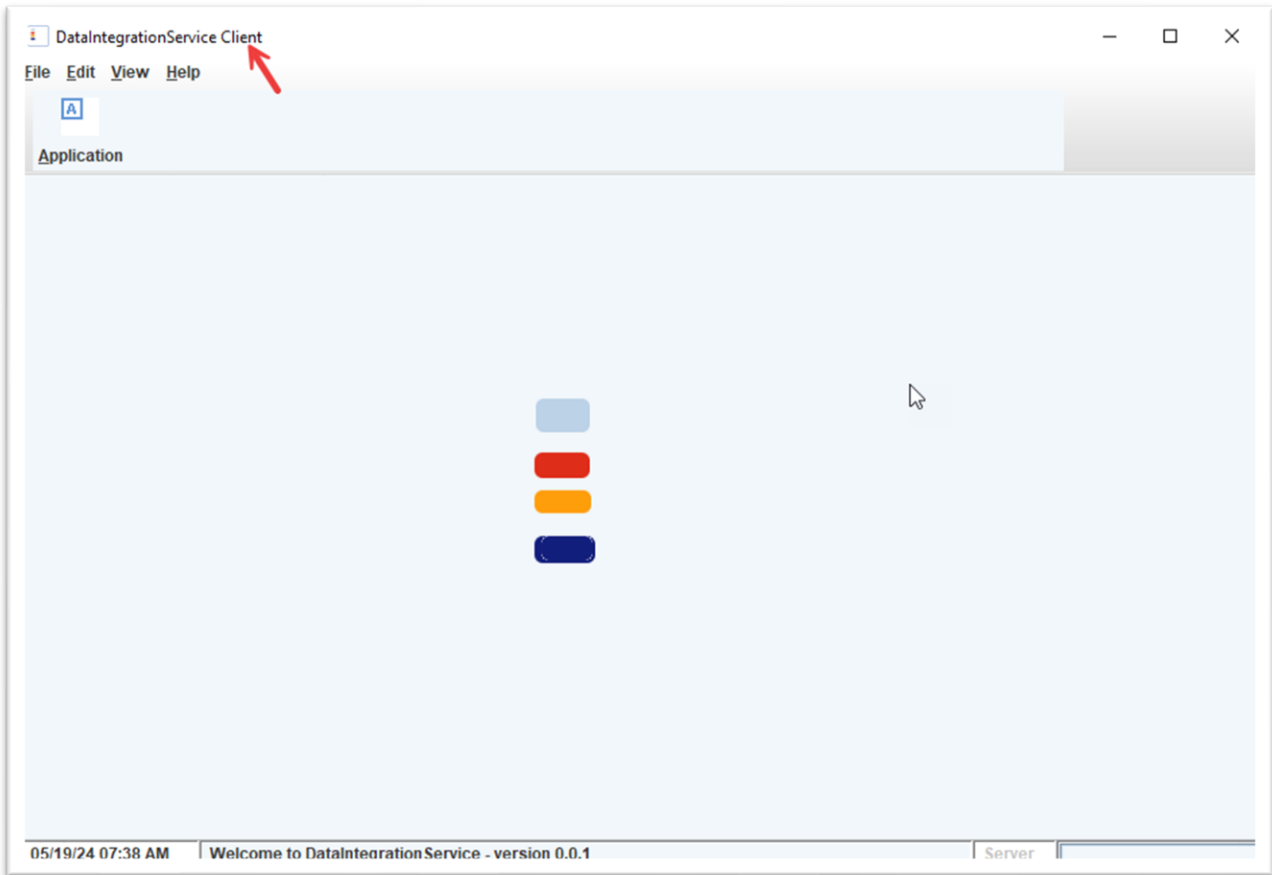
4. Select the project.
5. Click the “copy” button, which will create a new run configuration.
6. Rename to “client”.



5. Click the "Arguments" tab
6. Change to "client"
7. Change from "single" to "client"
8. Click "run" button

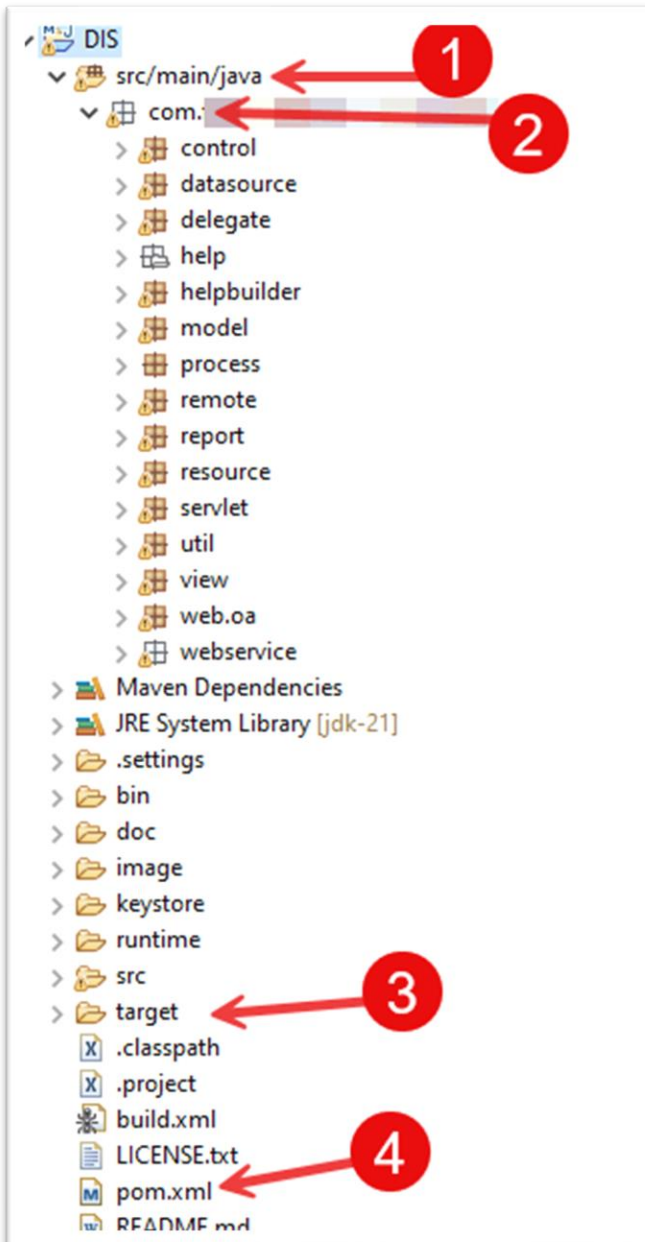


Login dialog for connecting to the application that is running as a “server” (above instructions). Click “ok” button.

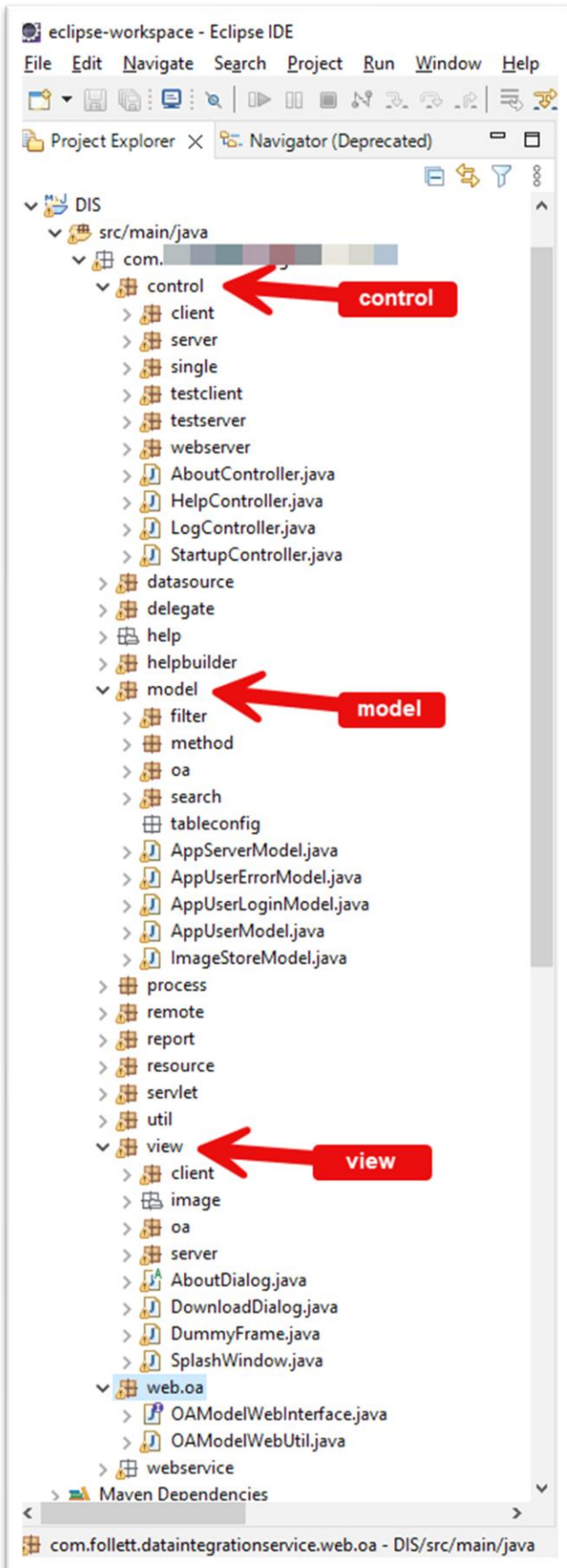


Window looks the same as running application as “single”, except it’s connected to server.
Run more than one instance to see how data stays in sync with the server and the other client applications.

Reviewing what was generated



1. Source code folder
2. Java Package
3. Target is where the complied classes are stored.
4. Maven pom file.



Follows MVC, “model / view / control”

“delegate” are helper methods for the Model Objects.

“datasource” has database mapping and ORM support.

“model” includes all of the Model Objects as OAOObject (/entity), and includes *Model classes that preconfigures objects together based on the relationships (/links).

“process” is used for Model Object processing.

“remote” extends how an application can run as multi-user (ex: client/server) or distributed.

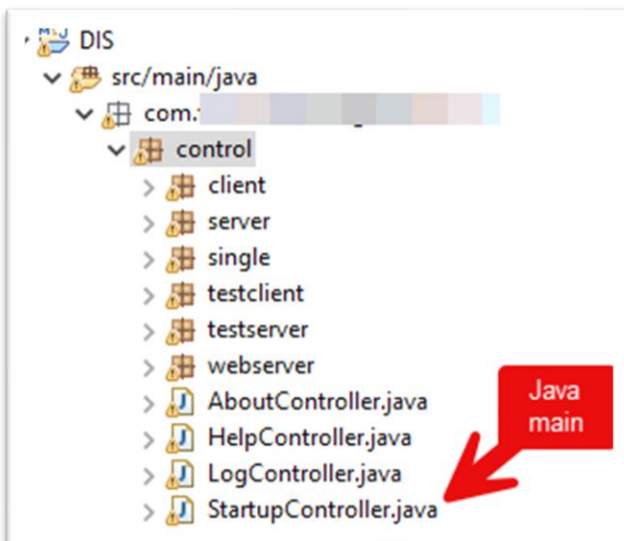
“report” for generating reports as html, pdf that can be used in a viewer or editor.

“resource” includes all of the properties for this project, listed in the values.properties.

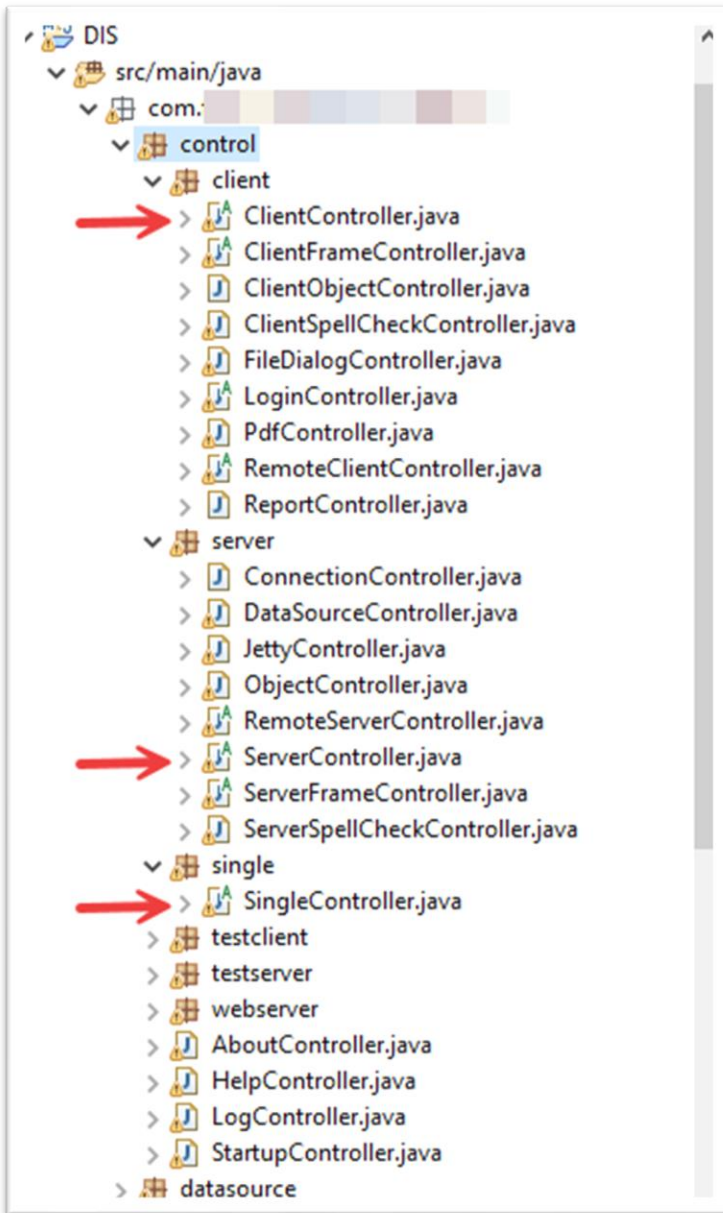
“view” for Java Swing/JFC desktop application.

“web” for web components.

Program Startup and Controllers



StartupController has the Java main method (application startup) and is responsible for receiving arguments. It requires an argument for application start type (single, server, client, webserver, webservice, test, etc). Depending on the start type, the matching *Controller will be called.



Each of the “type” controllers is then responsible for starting the other *Controllers.
 Example: type = “server” will call the ServerController, which then create it’s other controllers.
 JettyController manages the embedded Jetty webserver.
 DataSourceController manages datasources, databases and ORM.
 RemoteServerController manages multiplexer sockets for distributed communications.

Create new Version

Application has both a Version following “major.minor.patch”

The Release is the YYYYMMddX format.

The following need to be updated:

values.properties

pom.xml

Set version & release in version.ini in Appstore repo

Adding project to the OAppStore

To run from OAppStore, so that other users will have access.

Add to repo, jarstore Appstore

From OAppStore, create new AppType ...

qqqqqqqqqqqqqqqqqqqq

follow user guide for OAppStore

